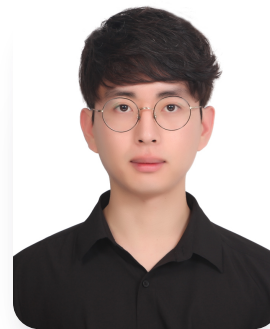


하흥주

복잡한 문제를 단순하게 추상화하는 프론트엔드 개발자

nusilite@gmail.com

문제를 정확히 정의하고, 단순한 해결책을 만듭니다



자기소개

레거시를 개선하고, 복잡한 문제를 추상화합니다

입시 학원에서 강사로 일하며 파이썬 CLI 기반의 진도 및 오답 관리 시스템을 만들었습니다. 그러나 CLI는 다른 강사들이 사용하기 어려웠고, 구글 시트와의 수동 동기화는 학생당 5분씩 시간을 소모했습니다. 이 레거시 시스템을 React 웹앱으로 전면 재설계했습니다.

웹앱 개발 과정에서 복잡한 문제를 단순하게 추상화하는 방법을 배웠습니다. 50개 이상의 셀을 가진 그리드의 상태 관리가 복잡해지자, Zustand로 컴포넌트별 로컬 스토어 패턴을 설계해 각 셀이 독립적으로 상태를 관리하도록 했습니다. 서버 응답이 7초 걸리는 문제는 DB와 API 서버의 네트워크 구조를 분석해 같은 리전에 배포하는 것으로 0.3초로 단축했습니다.

주도적으로 문제를 발견하고, 가설을 세워 해결책을 만드는 개발자입니다.

기술 스택

React, TypeScript, TanStack Query, Zustand, Vite, Git, Express, PostgreSQL, Prisma

경력

아르보 수학학원 중고등부 수학 | 강사 | 프리랜서

2020. 02. ~ 2024. 07. (4년 6개월)

- 파이썬과 구글 시트를 사용한 자체 제작 진도 및 오답 관리 시스템으로 체계적인 학생 관리
- 개념서 자체 제작, 100점 획득 학생 배출

아르보 수학학원 (부서 없음) | 개발자 | 계약직

2025. 11. ~ 2026. 04. (6개월)

진도 및 오답 관리 시스템 제작

담당 업무

- 레거시 시스템 개선
 - 기존 파이썬 CLI + 구글 시트 시스템을 React 웹앱으로 전면 재설계

- 수동 동기화(학생당 5분) → 자동화된 실시간 동기화로 전환
 - 복잡한 상태 관리 추상화
 - 50개 이상 셀을 가진 그리드 컴포넌트의 상태 관리 문제 해결
 - Zustand 로컬 스토어 패턴으로 각 셀이 독립적으로 상태 관리하도록 설계
 - 성과
 - 서버 응답 속도 단축 7초 → 0.3초 (22배 향상)
 - 그리드 인풋 반응 시간 단축 20.38ms → 2.28ms (89% 향상)
 - 서버 사이드 생성으로 PDF 생성 시간 단축 및 브라우저 부담 제거
 - 모바일 사용자 경험 고려한 반응형 구현 (랜딩, 회원가입, 권한 요청 페이지)
-

프로젝트

Arbor The Tree 오답관리 시스템 아르보 수학회원

2025. 11. ~ 2026. 04.

레거시 CLI 시스템을 웹앱으로 재설계

해결한 문제

- 레거시 파이썬 CLI 시스템의 한계
 - CLI 인터페이스로 인해 다른 강사들이 사용 불가
 - 진도표와 오답 과제 현황을 수기로 동기화 (학생당 5분, 수업당 50 100분 소요)
 - React 웹앱으로 전면 재설계하여 누구나 사용 가능한 시스템으로 전환
- 복잡한 그리드 상태 관리 추상화
 - 50개 이상 셀을 가진 그리드에서 각 셀의 상태 변경이 전체 리렌더링 유발
 - Zustand 로컬 스토어 패턴: 각 셀이 독립 스토어를 가지고 자신의 상태만 구독
 - 컴포넌트 간 결합도를 낮추고 상태 관리 복잡도를 단순화

주요 성과

- 서버 응답 속도 7초 → 0.3초로 단축 (22배 향상)
 - 문제 분석: Neon DB의 cold start와 region 불일치가 원인임을 가설로 설정
 - 해결: Railway PostgreSQL로 교체, API와 DB를 동일 internal network에 배포
- 그리드 입력 반응 시간 20.38ms → 2.28ms로 단축 (89% 향상)
 - 문제 분석: 50개 이상 셀의 동시 렌더링이 성능 저하 원인
 - 해결: TanStack Virtual 가상 스크롤 + Zustand 로컬 스토어로 DOM 최소화
- PDF 생성 시간 단축 및 브라우저 부담 제거
 - 서버 사이드 Typst 컴파일 도입으로 react-pdf 대비 80쪽 기준 30.7% 단축
 - 브라우저 메인 스레드 점유 없이 PDF 생성
- 모바일 사용자 경험 고려한 반응형 구현 (랜딩, 회원가입, 권한 요청 페이지)

기술적 해결

- 커스텀 배포 스크립트: npm install로 설치되지 않는 Typst compiler, font 파일을 Railway 배포 시 자동 설치
- RESTful API 설계: 프론트엔드와 분리된 [API 서버 구축](#)

Harpooning Tmux 세션 관리자 개인/기타

2026. 04. ~ 2026. 04.

어떤 프로젝트든 바로 열고 닫고 이동하기

해결한 문제

- 프로젝트 간 빠르게 이동을 하려고 Tmux를 사용했으나, 세션이 많아짐에 따라 원하는 세션을 찾는 데에 오랜 시간 소요
- 전 넷플릭스 개발자 ThePrimeagen이 만든 [harpoon](#)은 NeoVim에서의 즐겨찾기로 페이지를 등록, 관리, 이동하게 해주고 [tmux-sessionizer](#)는 컴퓨터 전체에서 프로젝트 폴더를 빠르게 검색하고 세션으로 추가하게 해줬으나, Tmux의 세션을 [harpoon](#)처럼 관리할 수 없음
- 즐겨찾기 목록에 적힌 세션 이름의 순서대로 단축키가 적용되게 하여 빠르게 세션 간 이동
- 즐겨찾기 목록과 실제 세션을 양방향 동기화

기술적 해결

- vim 바깥에서 vim motion을 사용할 수 있도록 tmux-popup 안에서 vim 실행
 - 마크다운의 체크박스 []를 [x]로 체크하고 종료하면 해당 세션을 선택할 수 있게 함
 - [grep](#)으로 체크된 줄을 찾고 [sed](#)로 체크박스를 제외한 세션 이름을 추출, 해당 세션으로 이동

주요 성과

- 새 프로젝트를 tmux 세션으로 등록하는 시간을 평균 25.3초에서 0.8초로 단축 (31배 향상)
- 세션 간 이동 시간 단축: 목록 순서대로 숫자 단축키(0 9)로 즉시 이동 가능
- 손쉬운 세션 정리: 목록 삭제만으로 실제 세션 종료 및 세션 종료시 즐겨찾기 목록에서도 삭제

포트폴리오

링크

[아르보 수학학원 오답 및 진도 관리 시스템](#)
[Harpooning Tmux 세션 관리자](#)

교육

고려대학교 졸업 | 대학교(학사) | 경제학과
2013. 02. ~ 2020. 02.